Allocatable Fortran wrappers, from 1D to N-D

Rohit Goswami · 2026-03-23 · python · fortran · numpy · f2py · ramblings

> *Half the work in computational science is moving data between programs that disagree about how memory works. Allocatable arrays are where the disagreement starts.*

Background

About a year ago I covered opaque pointers as a way to wrap non-`bind(c)` Fortran derived types without requiring more work from the Fortran standard. That post ended with a note about allocatable components. Since then, I've been working through a fix on the enh/f2py-derived-type-opaque branch (the code in this post corresponds to the post/f2py-allocatable-members tag).

Allocatable components have been part of Fortran since 2003 (F2018 7.5.4.7)[^fn:1], and they are everywhere in scientific code. A molecular dynamics trajectory stores positions as an allocatable 2D array. A finite element mesh stores connectivity the same way. Any type whose size is not known at compile time uses `allocatable`.

The question is how to get these arrays into and out of Python without scrambling the data. As with the opaque pointer post, we will build each layer by hand first, then show how `f2py` automates the whole thing.

The Fortran type

Let's start with something small. `DataVec` holds a tag and a dynamically-sized array:

```
module datavec_mod
  implicit none

  type :: DataVec
    integer :: tag
    real, allocatable :: values(:)
  end type DataVec

end module datavec_mod
```

Nothing fancy yet. The `allocatable` attribute means the array does not exist until someone allocates it, and the size can change at runtime. From Python we want `None` when unallocated, a NumPy array when allocated, and assignment to resize.

Manual Fortran wrappers

Following the same pattern as the `point` wrappers from the opaque pointer post, we need `bind(c)` accessor functions for the allocatable member. Four operations, each mechanically derived from the type definition:

▶ Full Fortran wrapper module (click to expand)

There isn't much to say about these. They are mechanically derived from the type, same as the `point` wrappers. The pattern repeats for every member: take a `c_ptr`, call `c_f_pointer` to get a typed Fortran pointer, do the operation, return a C-friendly result. The one thing that is new compared to scalar members

is `c_loc(obj%values)` on line 55, which requires F2008 or later[^fn:2].

We can test these from a Fortran driver to make sure the wrappers work before we touch C:

```
program test_datavec_wrappers
  use datavec_wrappers
  use iso_c_binding
  implicit none
  type(c_ptr) :: dv
  logical(c_bool) :: alloc
  integer :: n

  dv = datavec_create(42)
  alloc = datavec_values_allocated(dv)
  print *, "Allocated after create:", alloc  ! F

  ! We would call datavec_values_set here with a C array,
  ! but that requires a C caller. For now, verify the wrappers link.
  n = datavec_values_size(dv)
  print *, "Size:", n  ! 0

  call datavec_destroy(dv)
  print *, "Destroyed."
end program
```

```
$ gfortran datavec_mod.f90 datavec_wrappers.f90 test_wrappers.f90 -o test_wrappers
$ ./test_wrappers
 Allocated after create: F
 Size:           0
 Destroyed.
```

So far so good. The wrappers compile and the allocatable member starts unallocated.

Testing from C

The Fortran test could not exercise the setter (it needs a C array). So we write a C driver that exercises the full lifecycle:

▶ C test (click to expand)
